Deep RL for Automated Stock Trading

Weekly Progress Report — Week 10

Rutwik Shrikant Zhu Yufan Fei Xin

Semester 2, 2025/26

1 Summary

This week has focused on implementing the learning algorithms themselves. With the data pipeline and environment available, we have built DQN and DDQN agents from scratch in PyTorch and connected them to a full training loop.

2 DQN Implementation

We have implemented a feedforward Q-network that maps the environment state to Q-values for the discrete trading actions. The model uses a simple multilayer perceptron:

$$10 \to 128 \to 64 \to 3 \tag{1}$$

with ReLU activations.

We have also implemented the standard components required by DQN: a replay buffer, an online policy network, a target network, TD-loss optimisation with Adam, ε-greedy exploration, and gradient clipping. The replay buffer helps stabilise updates by reducing temporal correlation between transitions, while the target network makes TD targets less volatile during training. By the end of the week, the DQN pipeline has been able to collect experience, sample batches, and update Q-values consistently without crashing.

3 DDQN Extension

After the baseline DQN agent was working, we implemented Double DQN (DDQN). The main modification was to decouple action selection and target evaluation:

$$y_t = r + \gamma Q_{\bar{\theta}}(s', \arg\max_a Q_\theta(s', a)) \tag{2}$$

This is intended to reduce the overestimation bias commonly observed in vanilla DQN. The DDQN implementation reuses most of the DQN code and changes only the target calculation logic, which made it a natural extension once the first agent was functioning.

4 Training and Evaluation Pipeline

We have also built a complete training script that loads and splits a ticker dataset, initialises the environment and agent, runs training episodes, evaluates the current policy on validation data, compares against a buy-and-hold baseline, and saves trained model weights and plots.

At this stage, the main focus has been functional correctness rather than final performance. The goal this week has been to confirm that the agents can interact with the environment, accumulate transitions, update Q-values, and produce sensible portfolio traces. We have now reached the point where the full DQN/DDQN pipeline is operational end to end, even though the policies themselves still need substantial tuning.

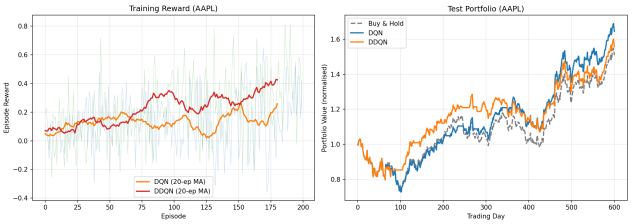

Figure 1: Example training/evaluation output generated after integrating DQN/DDQN with the environment.

5 Next Steps

With the core algorithms now working, the next step for the coming week is to improve performance. This will involve revisiting reward shaping, environment dy-

namics, evaluation correctness, checkpoint selection, and training stability so that the reported results better reflect the actual quality of the learned policy.